

Laboratorio #1 — Entrenamiento de Redes Neuronales (MLP de regresión)

1. Investigación: capas, pérdidas y optimizadores

nn.Linear. Capa densa que aplica $y = xW^T + b$. Parámetros: *in_features*, *out_features* y *bias*. La primera capa recibe las 8 features escaladas y la última tiene *out_features*=1 por tratarse de regresión.

Activaciones. Aportan la no linealidad; sin ellas, apilar capas densas equivale a una sola transformación lineal. *nn.ReLU*: $\max(0, x)$, barata y con gradiente estable en la zona positiva, aunque puede producir neuronas muertas. *nn.LeakyReLU*: introduce una pendiente pequeña en los negativos (*negative_slope*≈0.01) y mitiga ese problema. *nn.Tanh*: salida acotada en (−1,1) y centrada en cero, pero se satura en los extremos y frena el flujo del gradiente.

nn.Dropout. Regularización que apaga cada activación con probabilidad *p* durante el entrenamiento y se desactiva en evaluación (`model.eval()`). Reduce la coadaptación de neuronas.

nn.BatchNorm1d. Normaliza cada feature dentro del mini-batch (media≈0, varianza≈1) y aplica un escalado y desplazamiento aprendibles. Su parámetro *num_features* debe coincidir con la salida de la capa anterior. Estabiliza y suele acelerar la convergencia.

Pérdidas de regresión. *nn.MSELoss* promedia $(y - y_{\text{pred}})^2$, penaliza fuerte los errores grandes y es sensible a outliers; *nn.L1Loss* promedia $|y - y_{\text{pred}}|$ y es más robusta; *nn.SmoothL1Loss* (Huber) se comporta como L2 cerca de cero y como L1 lejos, con umbral *beta*. En este laboratorio se entrenó con MSELoss por coherencia con la métrica principal.

Optimizadores. El *lr* (learning rate) fija el tamaño del paso: demasiado alto vuelve el entrenamiento inestable y demasiado bajo lo deja corto en el presupuesto de épocas. El *weight_decay* aplica regularización L2 penalizando pesos grandes. *SGD* actualiza en dirección opuesta al gradiente (aquí con *momentum*=0.9) y no adapta el paso por parámetro, por lo que exige afinar bien el lr. *Adam* mantiene estimaciones de primer y segundo momento y adapta el paso por parámetro, lo que le da buena convergencia sin ajuste fino. *RMSprop* adapta el lr con un promedio móvil de gradientes al cuadrado, sin el momento de primer orden de Adam; funciona bien con gradientes ruidosos.

2. Datos y configuración experimental

Se usó *California Housing* (20 640 observaciones, 8 features numéricas y el target *MedHouseVal*). No hay nulos ni duplicados; sí hay outliers claros en *AveOccup* (máx. ≈1243 frente a mediana ≈2.8) y un target censurado en 5.0, que se conservaron. Todas las variables son numéricas, por lo que no hubo codificación categórica. Se aplicó *StandardScaler* ajustado solo con train, y un split 60/20/20 (12 384 / 4 128 / 4 128) con *random_state*=42. El test se reservó para una única evaluación final. Se entrenaron 12 configuraciones partiendo de un baseline ([64,32], ReLU, Adam lr=1e-3, batch 32, 40 epochs, sin regularización), variando 1–2 factores a la vez para poder atribuir el efecto de cada cambio.

3. Tabla resumen de las 12 iteraciones (métricas sobre validación)

#	Arquitectura	Activación	Optimizador / LR	Batch / Ep.	Regularización	MSE (val)	MAE (val)	RMSE (val)
1	[64, 32]	ReLU	Adam / 0.001	32 / 40	—	0.2945	0.3760	0.5427
2	[128, 64]	ReLU	Adam / 0.001	32 / 40	—	0.2858	0.3559	0.5346
3	[64, 64, 32]	ReLU	Adam / 0.001	32 / 40	—	0.3015	0.3661	0.5491
4	[64, 32]	LeakyReLU	Adam / 0.001	32 / 40	—	0.2890	0.3701	0.5376
5	[64, 32]	Tanh	Adam / 0.001	32 / 40	—	0.2898	0.3712	0.5383

#	Arquitectura	Activación	Optimizador / LR	Batch / Ep.	Regularización	MSE (val)	MAE (val)	RMSE (val)
6	[64, 32]	ReLU	SGD / 0.01	32 / 40	—	1.3767	0.9171	1.1733
7	[64, 32]	ReLU	RMSprop / 0.001	32 / 40	—	0.2862	0.3572	0.5350
8	[64, 32]	ReLU	Adam / 0.0001	32 / 40	—	0.3578	0.4192	0.5982
9	[64, 32]	ReLU	Adam / 0.005	32 / 40	—	0.2882	0.3601	0.5369
10	[64, 32]	ReLU	Adam / 0.001	32 / 40	dropout=0.2	0.2896	0.3663	0.5381
11	[64, 32]	ReLU	Adam / 0.001	32 / 40	L2 wd=1e-4 + BatchNorm	0.6229	0.5319	0.7892
12	[64, 32]	ReLU	Adam / 0.001	64 / 60	—	0.2901	0.3622	0.5386

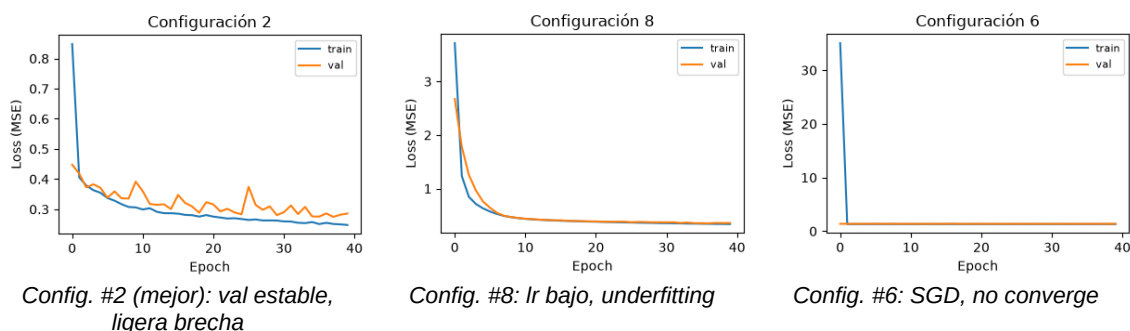
4. Mejor modelo y evaluación final en test

La configuración #2 ([128, 64], ReLU, Adam con $\text{lr}=0.001$, batch 32, 40 epochs, sin regularización) obtuvo el menor MSE de validación (0.2858). Solo ese modelo se evaluó una única vez sobre el conjunto de test:

Conjunto	MSE	MAE	RMSE
Validación	0.2858	0.3559	0.5346
Test (evaluación única)	0.2809	0.3532	0.5300

El test resulta levemente mejor que validación (MSE 0.2809 vs. 0.2858), lo que indica que la selección de hiperparámetros no sobreajustó al conjunto de validación y que el modelo generaliza de forma consistente. En unidades del target, el RMSE de 0.5300 equivale a un error de aproximadamente 53 000 USD y el MAE de 0.3532 a unos 35 300 USD.

5. Curvas de pérdida



6. Discusión y análisis

¿Qué cambio tuvo el mayor impacto positivo y cuál el mayor impacto negativo?

El mayor impacto positivo fue **ampliar la arquitectura** de [64, 32] a [128, 64] (config. #2): el MSE de validación bajó de 0.2945 a 0.2858 (−3.0 %) y el MAE de 0.3760 a 0.3559 (−5.3 %). Le siguió el cambio de optimizador a RMSprop (#7, MSE 0.2862), prácticamente empatado. El mayor impacto negativo fue **cambiar a SGD con $\text{lr}=0.01$** (config. #6), que llevó el MSE a 1.3767, casi 4.7 veces el baseline; el segundo peor fue la combinación **L2 + BatchNorm** (#11, MSE 0.6229) y el tercero **$\text{lr}=1\text{e-}4$** (#8, MSE 0.3578). El patrón es claro: en este dataset la elección del optimizador y del learning rate pesa mucho más que la arquitectura, cuyos cambios se movieron todos dentro de un rango estrecho (0.2858–0.3015).

¿Observó overfitting o underfitting? ¿Cómo lo identificó y cómo lo mitigó?

Se observaron ambos. En la config. #2 la pérdida final de entrenamiento (0.2479) queda por debajo de la de validación (0.2858): hay una brecha, pero la curva de validación se mantiene plana y no repunta, así que es un **overfitting leve y controlado**. El **underfitting** aparece en las config. #8 y #6: en #8 ambas curvas quedan altas y descendiendo muy lentamente porque $\text{lr}=1\text{e-}4$ no alcanza a converger en 40 epochs, y en #6 la curva de SGD se estanca en un nivel muy alto. La mitigación probada fue dropout=0.2 (#10), que efectivamente cerró la brecha train–val y mejoró sobre el baseline (0.2945 → 0.2896), aunque sin superar a #2; para #8 y #6 la corrección

correcta no es regularizar sino subir el learning rate o aumentar las épocas.

¿La regularización mejoró el desempeño en validación?

Solo marginalmente y no en todos los casos. **Dropout 0.2** (#10, MSE 0.2896) mejoró frente al baseline (0.2945), por lo que fue la mejor de las estrategias de regularización probadas; en cambio **L2 (wd=1e-4) combinada con BatchNorm** (#11) empeoró drásticamente el resultado (0.6229). La explicación es que el overfitting aquí es muy pequeño: con 12 384 muestras de entrenamiento y redes de solo dos capas ocultas el modelo no llega a memorizar, así que la regularización principalmente resta capacidad. En el caso de #11 se cambiaron dos factores a la vez, y BatchNorm con batches de 32 sobre datos ya estandarizados añade ruido en las estadísticas del mini-batch, lo que degradó la convergencia más de lo que aportó el weight_decay. La mejor configuración global (#2) no usa regularización alguna.

¿Cómo se relacionan batch size y epochs con la estabilidad y la velocidad?

La config. #12 (batch 64 y 60 epochs frente a 32 y 40 del baseline) obtuvo MSE 0.2901, apenas mejor que el baseline y aún por detrás de #2. Un batch más grande produce menos actualizaciones por época, con gradientes promediados sobre más muestras: las curvas resultan más suaves y cada época es más rápida, pero el avance por época es menor, y por eso hizo falta subir a 60 epochs para compensar. Que ese aumento no produjera una mejora apreciable indica que el modelo ya había convergido alrededor de la época 40 con batch 32; entrenar más tiempo solo agrega costo sin ganancia.

Comparando MSE, MAE y RMSE, ¿qué muestra cada métrica?

El **MSE** (0.2809 en test) eleva los errores al cuadrado, por lo que queda dominado por los casos extremos —en particular las viviendas censuradas en el tope de 5.0— y es útil como función objetivo por ser derivable y castigar fuerte los fallos grandes, pero su valor no es interpretable en unidades del target. El **RMSE** (0.5300) devuelve esa escala y permite leer el error en cientos de miles de USD. El **MAE** (0.3532) representa el error típico sin amplificar los extremos. La relación RMSE/MAE ≈ 1.5 revela que la distribución de errores tiene cola pesada: la mayoría de predicciones falla cerca de 0.35, pero unas pocas fallan mucho más. Esta diferencia también permite distinguir modelos que el MSE por sí solo confundiría: las config. #2 y #4 tienen MSE casi idéntico (0.2858 vs 0.2890) pero MAE claramente distinto (0.3559 vs 0.3701), es decir, #4 falla algo más en el caso promedio.

¿Qué elegiría para un modelo de producción?

Elegiría la config. #2 como punto de partida: **[128, 64] con ReLU, Adam con lr=0.001, batch 32** y alrededor de 40 epochs, agregando *early stopping* sobre el MSE de validación para no fijar el número de épocas a mano, y evaluando dropout ligero (0.1–0.2) dado que fue la única regularización que aportó algo. Para seguir optimizando usaría **búsqueda bayesiana** (por ejemplo con Optuna): el grid search ordenado ya cumplió su función de mapear qué ejes importan, pero crece de forma combinatoria, y el random search desperdicia entrenamientos en zonas que estos resultados ya descartan (SGD, lr muy bajos). La búsqueda bayesiana concentra las evaluaciones alrededor de la región prometedora —Adam o RMSprop con lr entre 1e-3 y 5e-3 y capas anchas— que es lo que conviene cuando cada entrenamiento tiene un costo apreciable.

7. Conclusiones

El mejor MLP alcanzó un RMSE de 0.5300 en test ($\approx 53\,000$ USD de error), con métricas de test consistentes con las de validación, lo que confirma que el protocolo de una sola evaluación final funcionó. El optimizador y el learning rate fueron los hiperparámetros determinantes: Adam y RMSprop convergieron sin ajuste fino mientras que SGD con lr=0.01 fracasó por completo. La arquitectura influyó poco y ampliar la red dio más resultado que profundizarla, ya que la tercera capa (#3) empeoró el MSE. Finalmente, la regularización no era necesaria para este tamaño de red y de dataset: el mejor modelo no usa ninguna, y añadir L2 con BatchNorm resultó contraproducente. Como limitación, los outliers de *AveOccup* y el truncamiento del target en 5.0 se conservaron sin tratamiento, y la brecha entre RMSE y MAE sugiere que ahí se concentra buena parte del error residual; un recorte por percentiles ajustado solo con train sería el siguiente experimento, junto con repetir cada configuración con varias semillas para separar el efecto de los hiperparámetros de la variabilidad de la inicialización.